

# Relatório de Desenvolvimento: Módulo AVD1 - ASP.NET Core MVC

---

## Identificação

---

**Aluno:** João Victor Dantas Cotrim

**Matrícula:** 202311638

**Curso:** Sistema de Informação

**Disciplina:** Desenvolvimento Web (Módulo AVD1)

**Professor:** Rosenclever

---

## Introdução

---

Este relatório apresenta o desenvolvimento dos exemplos e a resolução dos exercícios propostos nos slides do módulo AVD1, que aborda a introdução ao ASP.NET Core MVC e a construção de um CRUD completo. O documento está organizado de acordo com os arquivos PDF fornecidos, detalhando os enunciados e as respectivas respostas para os exercícios teóricos, bem como a indicação dos exercícios práticos desenvolvidos. O código-fonte resultante das atividades práticas encontra-se versionado e disponível em um repositório no GitHub.

---

## Desenvolvimento

---

Nesta seção, são apresentados os exercícios extraídos de cada um dos arquivos de material didático, indicando o slide correspondente, o enunciado e a solução.

---

## Arquivo: 1 - Introdução ao ASP .NET Core.pdf

### Slide 36 - Exercício Prático

**Enunciado:** Utilize seus conhecimentos de CSS e HTML para que a página de Registro de Alunos pareça com a da figura.

**Desenvolvimento:** Foi criada a classe `Aluno` com os atributos solicitados (Nome, Email, Telefone, Endereco, Complemento, Bairro, Municipio, Uf, Cep). Em seguida, foi implementado o `AcademicoController` e a View `Aluno.cshtml`, utilizando as classes do Bootstrap (`form-group`, `form-control`, `panel`) para estruturar o formulário de cadastro de alunos conforme o modelo visual apresentado nos slides.

---

## Arquivo: 2 - CRUD Aluno Sem Template.pdf

### Slides 9 a 24 - Exercícios Práticos

**Enunciado:** Implementar as Views e Controllers para o cadastro e listagem de alunos, utilizando Tag Helpers (`asp-for`, `asp-action`) e o padrão DAO.

#### Desenvolvimento:

- **Formulário de Cadastro:** Ajustado para utilizar `@model Aluno` e as Tag Helpers `asp-for` em cada campo.
  - **Listagem (Index):** Criada a View `Index.cshtml` para exibir os alunos cadastrados em uma tabela HTML, utilizando `@model IEnumerable<Academico.Models.Aluno>`.
  - **Controller:** O `AlunoController` foi atualizado para incluir os métodos `Create` (GET e POST) e `Index`, utilizando as anotações `[HttpPost]`, `[ValidateAntiForgeryToken]` e `[Bind]`.
  - **Padrão DAO:** Criada a classe `AlunoDao` para encapsular o acesso aos dados (em memória, utilizando uma lista estática), contendo os métodos `GravarAluno` e `ObterTodos`.
-

## Arquivo: 3 - CRUD Aluno sem Template - PArte 2.pdf

### Slide 27 - Exercícios Teóricos (Fixação I)

**Enunciado:** Utilizando Data Annotations faça o que se pede:

1. Ajuste as mensagens de erro tendo por base as boas práticas de UX.
2. Como fazer com que o Complemento seja opcional?
3. Como limitar a entrada de dados para o Telefone e o CEP?
4. Como fornecer uma máscara para a entrada de dados de Telefone e CEP?

### Respostas:

1. Para ajustar as mensagens de erro, basta usar a propriedade `ErrorMessage` dentro da anotação. Exemplo: `[Required(ErrorMessage = "O campo Nome é obrigatório.")]`.
2. Para o Complemento ser opcional, é só não colocar o `[Required]` nele. Assim o sistema não vai exigir o preenchimento.
3. Para limitar a entrada de dados, usa-se o `[StringLength]`. Exemplo para o CEP: `[StringLength(9, MinimumLength = 8)]`.
4. O Data Annotations sozinho não coloca máscara visual na tela. Para isso precisa usar JavaScript (tipo jQuery Mask) no HTML. Mas para validar o formato no back-end, usa-se `[RegularExpression(@"^\d{5}-\d{3}$")]`.

---

### Slide 28 - Exercícios Teóricos (Responda)

**Enunciado:**

1. O que vem a ser sobrecarga dentro da POO?
2. Onde foi possível observar a aplicação do conceito de sobrecarga no código que estamos desenvolvendo?
3. Por que a lista de alunos teve que ser definida como estática em nossa aplicação? O que aconteceria com o funcionamento do programa, caso a propriedade estática fosse removida de lista alunos?
4. Qual o significado da sigla CRUD?

5. Se os métodos possuem o mesmo nome na mesma classe, como por exemplo o método `Edit`, como o compilador consegue decidir qual método deve ser acionado quando os dados são enviados a partir de um formulário?
6. Por que o método `Delete` de `AlunoController` teve que ter seu nome alterado para `DeleteConfirmed` diferentemente do que aconteceu com o `Edit` que pode ter sobrecarga?

## Respostas:

1. Sobrecarga é quando a gente tem métodos com o mesmo nome na mesma classe, mas com parâmetros diferentes.
2. No método `Edit` do Controller. Tem um `Edit(int id)` que abre a tela de edição e um `Edit(int id, Aluno aluno)` que salva os dados alterados.
3. A lista foi colocada como `static` para os dados não sumirem entre as requisições. Se tirasse o `static`, toda vez que o usuário acessasse uma página, a lista ia ser criada do zero e os alunos cadastrados iam sumir.
4. CRUD significa **C**reate (Criar), **R**ead (Ler), **U**ppdate (Atualizar) e **D**elele (Excluir). São as quatro operações básicas de um sistema.
5. O ASP.NET diferencia pelo verbo HTTP. O método que mostra a tela responde ao `GET`, e o que recebe os dados do formulário tem o atributo `[HttpPost]`, então responde ao `POST`.
6. Porque os dois métodos (mostrar a tela de exclusão e confirmar a exclusão) iam ter a mesma assinatura: `Delete(int id)`. Como não pode ter dois métodos iguais, o segundo foi chamado de `DeleteConfirmed` e recebeu o atributo `[ActionName("Delete")]` para o framework saber que ele é a action `Delete` para `POST`.

---

## Slides 20 e 23 - Exercícios Práticos

**Enunciado:** Criar as Views `Details` e `Delete` utilizando o assistente de criação do Visual Studio e implementar os respectivos métodos no Controller.

**Desenvolvimento:** Foram implementados os métodos `Details(int id)`, `Delete(int id)` (GET) e `DeleteConfirmed(int id)` (POST) no `AlunoController`. As

Views correspondentes foram geradas para exibir os dados do aluno em modo de leitura e para solicitar a confirmação antes da exclusão do registro.

---

## Arquivo: 4 - Desacoplando a classe Aluno.pdf

### Slides 3 a 12 - Exercícios Práticos

***Enunciado:** Implementar o padrão Repository para desacoplar o código, criando a interface `IALunoRepository`, a classe `InMemoryAlunoRepository` e ajustando o `AlunoController` para utilizar injeção de dependência.*

#### Desenvolvimento:

- **Interface:** Criada a interface `IALunoRepository` contendo as assinaturas dos métodos assíncronos ( `GetAll` , `GetById` , `Create` , `Edit` , `Delete` , `Exists` ).
- **Implementação:** Criada a classe `InMemoryAlunoRepository` que implementa a interface, gerenciando a lista de alunos de forma *thread-safe* (utilizando `lock` ).
- **Injeção de Dependência:** O `AlunoController` foi refatorado para receber `IALunoRepository` em seu construtor. Todos os métodos do CRUD foram atualizados para serem assíncronos ( `async Task<IActionResult>` ) e utilizarem os métodos do repositório.
- **Configuração:** O repositório foi registrado no contêiner de injeção de dependência no arquivo `Program.cs` utilizando o padrão Singleton ( `builder.Services.AddSingleton<IALunoRepository, InMemoryAlunoRepository>();` ).

---

## Repositório do Projeto

O código-fonte completo contendo a implementação de todos os exercícios práticos descritos neste relatório está disponível no seguinte repositório do GitHub:

**Link do Repositório:** <https://github.com/Noitah/AVD1>

---

## Conclusão

---

O desenvolvimento das atividades propostas no módulo AVD1 permitiu a compreensão prática e teórica dos fundamentos do ASP.NET Core MVC. A evolução do projeto, desde a criação de um formulário simples até a implementação de um CRUD completo com validações (Data Annotations) e boas práticas de arquitetura (padrão Repository e Injeção de Dependência), demonstrou a robustez do framework. A separação de responsabilidades e o desacoplamento do código garantem uma aplicação mais organizada e preparada para futuras integrações com bancos de dados reais, como o Entity Framework.

---

## Referências

---

1. MICROSOFT. **Introdução ao ASP.NET Core MVC.** Disponível em: <https://learn.microsoft.com/pt-br/aspnet/core/tutorials/first-mvc-app/start-mvc>. Acesso em: fevereiro de 2025.
2. MICROSOFT. **Validação com validadores de dados.** Disponível em: <https://learn.microsoft.com/pt-br/aspnet/mvc/overview/older-versions-1/models-data/validation-with-the-data-annotation-validators-cs>. Acesso em: fevereiro de 2025.
3. BYTEHIDE. **Data Annotation in C#: your complete guide.** Disponível em: <https://www.bytehide.com/blog/data-annotations-in-csharp>. Acesso em: fevereiro de 2025.
4. DIO. **O que é CRUD?** Disponível em: <https://www.dio.me/articles/o-que-e-crud>. Acesso em: fevereiro de 2025.
5. MICROSOFT. **Injeção de dependência no ASP.NET Core.** Disponível em: <https://learn.microsoft.com/pt-pt/aspnet/core/fundamentals/dependency-injection>. Acesso em: março de 2026.
6. MICROSOFT. **Implementando o repositório e a unidade de padrões de trabalho em um aplicativo MVC ASP.NET.** Disponível em: <https://learn.microsoft.com/pt-br/aspnet/mvc/overview/older-versions/getting-started-with-ef-5-using-mvc-4/implementing-the-repository-and-unit-of-work-patterns-in-an-asp-net-mvc-application>. Acesso em: março de 2026.